Predictive Maintenance | Final Project Report

Learner: ManjunathaNS
Project: Capstone Project 3 - Predictive Maintenance
Focus: ML deployment, automation, evaluation and business recommendations

Executive Summary

This final project builds an end-to-end predictive maintenance workflow for estimating machine failure risk from operating telemetry. The solution registers and validates data, performs EDA, prepares train/test artifacts, tunes multiple models, saves the best model, provides a Streamlit deployment app and automates the ML steps with GitHub Actions.

The selected model is **Bagging**. It achieved test F1 **0.784**, precision **0.860**, recall **0.721** and ROC-AUC **0.967**. Business users should treat predictions as risk scores for inspection prioritisation, not automatic shutdown decisions.

1. Data Registration

Methodology

Created the master folder with a data subfolder, placed the AI4I 2020 CSV in it and registered it through a validation script that checks expected columns, missing values, duplicates, types, target balance and machine type distribution.

Metric	Value
Rows	10000
Columns	14
Missing values	0
Duplicate rows	0
Failures	339
Failure rate	3.39%

Table 1. Registered dataset summary

Source: AI4I 2020 Predictive Maintenance Dataset, UCI Machine Learning Repository, DOI: https://doi.org/10.24432/C5HS5C.

2. Exploratory Data Analysis

Methodology

The EDA reviewed data background, row/column overview, univariate sensor distributions, bivariate comparisons by failure status, multivariate correlations and operational observations.

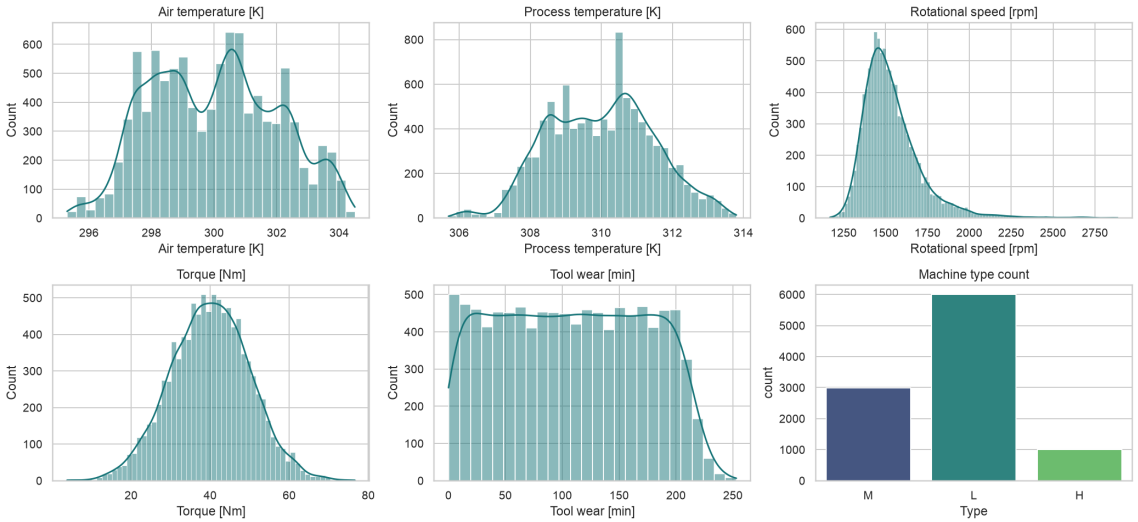


Figure 1. Univariate distributions of machine telemetry and type.

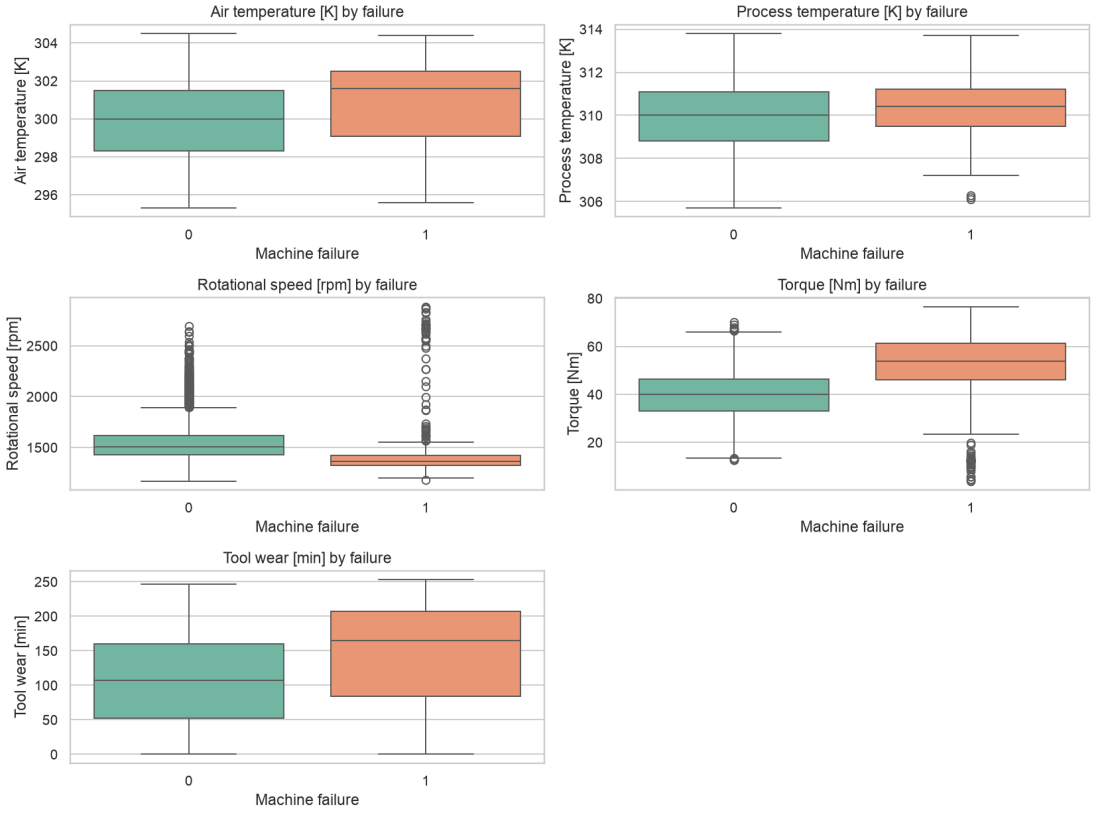


Figure 2. Bivariate sensor distributions by machine failure label.

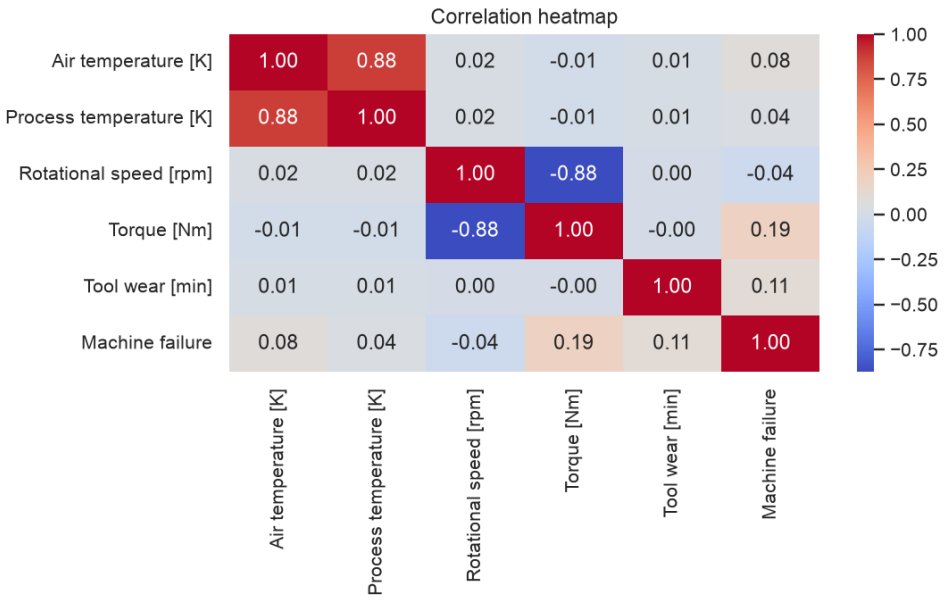


Figure 3. Correlation heatmap across sensor measurements and target.

Key insight: failures are rare (3.39%), so a high-accuracy model can still miss important failures. Evaluation therefore emphasises F1, recall and confusion-matrix review. Type L is the largest group and has a higher observed failure rate than H and M in this dataset; deployment monitoring should segment model performance by machine type.

3. Data Preparation

Methodology

Loaded the dataset from the repository data folder, removed identifiers (UDI and Product ID), removed failure-mode columns (TWF, HDF, PWF, OSF and RNF) to avoid target leakage, standardised column names and created a stratified 80/20 split saved as local CSV files.

Artifact	Rows	Purpose
data/cleaned.csv	10,000	Cleaned modelling table
data/train.csv	8,000	Training input artifact
data/test.csv	2,000	Held-out evaluation artifact

Table 2. Prepared train/test artifacts

The GitHub Actions workflow uploads train/test splits as the `train-test-splits` artifact and the downstream job downloads it for model training, satisfying workflow artifact handoff requirements.

4. Model Building with Experimentation Tracking

Methodology

Trained Decision Tree, Bagging, Random Forest, AdaBoost, Gradient Boosting and XGBoost models. Each candidate used GridSearchCV with five-fold cross-validation and F1 scoring. The code logs search spaces, best parameters, cross-validation F1, test metrics and confusion matrices to models/metrics.json, then saves the selected model to models/best_model.joblib.

Model	CV F1	Precision	Recall	Test F1	ROC-AUC
Decision Tree	0.701	0.836	0.676	0.748	0.874
Bagging	0.776	0.860	0.721	0.784	0.967
Random Forest	0.656	0.897	0.515	0.654	0.972
Adaboost	0.477	0.667	0.412	0.509	0.955
Gradient Boosting	0.743	0.873	0.706	0.780	0.968
Xgboost	0.746	0.887	0.691	0.777	0.977

Table 3. Model tuning and held-out performance

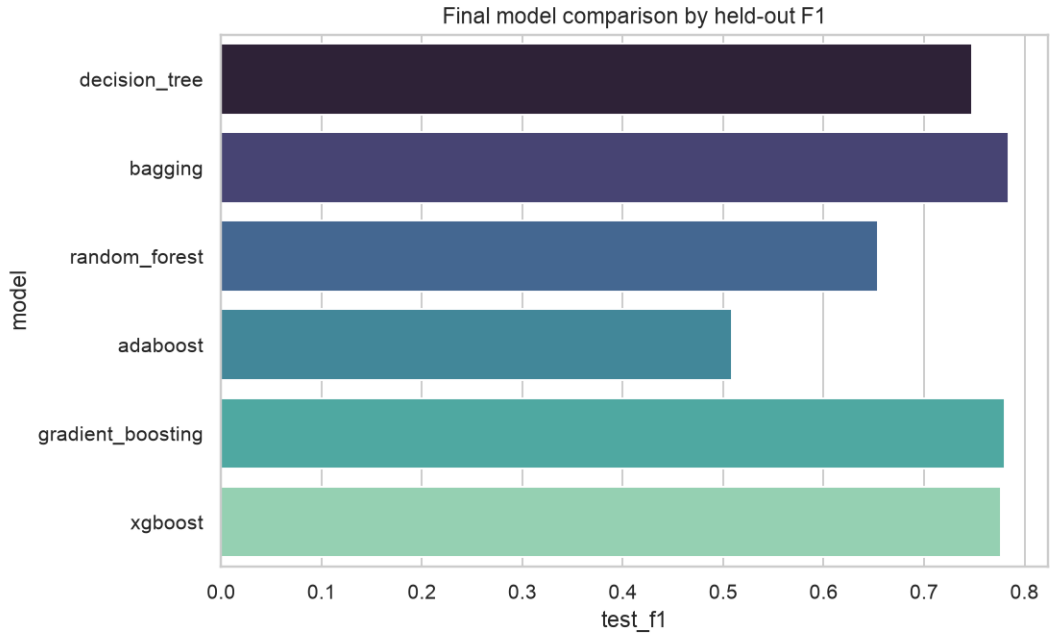


Figure 4. Held-out model comparison by F1 score.

The selected model predicts 49 of 68 failures in the held-out split and misses 19 failures. Business interpretation: the model is useful for prioritising inspections, but the alert threshold should be tuned using the cost of downtime versus false alarms.

5. Model Deployment

Methodology

Created a Streamlit app (`app.py`) that loads the committed `models/best_model.joblib`, collects machine type and five numeric operating inputs, stores them in a dataframe and returns failure probability plus a risk message. Deployment dependencies are listed in `requirements.txt`.

Streamlit Community Cloud Evidence

```
Deployment app file: app.py
Dependency file: requirements.txt
Main model loaded: models/best_model.joblib
Inputs captured into a dataframe:
  Type, Air_temperature, Process_temperature, Rotational_speed, Torque, Tool_wear
Output displayed:
  Failure probability and low/high risk recommendation
Deployment URL: add after Streamlit Community Cloud deployment.
```

Figure 5. Streamlit deployment app evidence and expected UI inputs.

Requirement	Status
App loads committed model	Implemented in app.py
Inputs saved into dataframe	Implemented and displayed in app
Dependency file	requirements.txt
Community Cloud deployment	User must deploy from GitHub after push

Table 4. Deployment checklist

6. Automated GitHub Actions Workflow

Methodology

The pipeline automates validation, cleaning, train/test artifact upload, artifact download, model tuning, Streamlit smoke test, trained-model artifact upload and commit of generated outputs back to main when files change.

Automated GitHub Actions Workflow

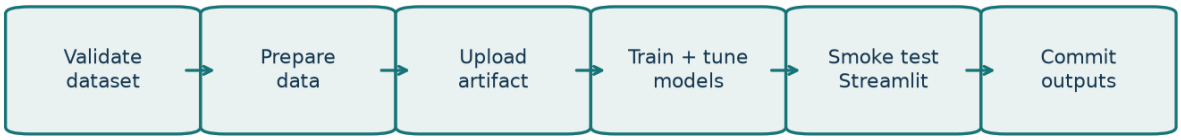


Figure 6. Automated ML workflow from validation through committed outputs.

GitHub Repository Folder Structure

```
predictive_maintenance_final/
├── .github/workflows/pipeline.yml
├── app.py                          # Streamlit app
├── data/ai4i2020.csv               # registered dataset
├── data/train.csv, data/test.csv  # workflow artifacts
├── models/best_model.joblib        # committed model
├── models/metrics.json             # experiment tracking log
├── notebooks/Predictive_Maintenance_Final_Notebook.ipynb
├── reports/figures/
├── src/validate_dataset.py
├── src/prepare_data.py
└── src/train.py
```

Figure 7. Repository folder structure evidence.

GitHub Actions Evidence

```
Status after push: to be verified after pushing project-18 repository.
Expected jobs:
  ✓ validate-prepare
  ✓ train-evaluate-deploy-check
Expected artifacts:
  • train-test-splits
  • trained-model-and-metrics
Expected commit behavior:
  • Generated train/test/model/metrics are committed back to main if changed.
```

Figure 8. GitHub workflow run evidence placeholder.

After pushing to GitHub, replace the placeholder workflow evidence with the real run screenshot if desired. The YAML file already contains the complete automated workflow steps required by the rubric.

7. Output Evaluation

Repository evidence should include the GitHub repository URL, folder structure screenshot and successful workflow run screenshot. Streamlit evidence should include the deployed app URL and app screenshot. This report includes generated evidence diagrams and placeholders; after the user deploys to Streamlit Community Cloud, the final URL and screenshot should be added if required by the submission portal.

Evidence item	Current status
GitHub repository	Ready to push from project-18
Executed workflow	Workflow YAML ready; run after push
Streamlit app	app.py ready; deploy on Streamlit Cloud
Submission files	Generated PDF and HTML notebook

Table 5. Output evidence tracker

8. Actionable Insights and Recommendations

1. Use the model as a risk-ranking tool for maintenance scheduling rather than a hard replacement for operator judgement.
2. Monitor missed failures closely because false negatives create downtime and repair cost.
3. Tune the probability threshold with business cost inputs: planned inspection cost, false alarm cost and unplanned downtime cost.
4. Re-train and validate on real historical machine data before production usage because the AI4I data is synthetic.
5. Add drift checks for torque, tool wear, rotational speed and temperature distributions after deployment.

Appendix

Raw code and full outputs are provided in the executed HTML notebook. Supporting files include src/validate_dataset.py, src/prepare_data.py, src/train.py, app.py, .github/workflows/pipeline.yml, models/metrics.json and models/best_model.joblib.

Representative tuning output: {'decision_tree': {'max_depth': 8, 'min_samples_split': 2}, 'bagging': {'max_samples': 1.0, 'n_estimators': 100}, 'random_forest': {'max_depth': None, 'n_estimators': 200}, 'adaboost': {'learning_rate': 1.0, 'n_estimators': 100}, 'gradient_boosting': {'learning_rate': 0.05, 'max_depth': 5, 'n_estimators': 200}, 'xgboost': {'learning_rate': 0.05, 'max_depth': 5, 'n_estimators': 200}}